

# Rapport Intermédiaire Projet ATRSC

## Gestion Optimisée d'Atelier

Antoine Chosson

Arthur Ollivier

### Question 1

Une condition à l'existence d'un régime permanent est que le flux entrant imposé au système reste inférieur à sa capacité de traitement. Les ouvriers ici sont ceux qui subissent le flux et le traite à la vitesse imposée par la machine sur laquelle ils travaillent. On vérifie que la charge de travail  $\rho$  de chaque employé est  $< 1$ . Si le flux d'entrée est trop grand, les files d'attentes divergent et le régime permanent n'est pas atteint.

Par un raisonnement heuristique, on approxime le temps de traitement de chaque produit sur chaque machine par l'espérance  $\mathbb{E}[S]$  de la loi uniforme associée.

| Machines |    | M1    | M2    | M3    | M4    | M5    | M6    | M7    | M8    |
|----------|----|-------|-------|-------|-------|-------|-------|-------|-------|
| Types    | T1 | 0,68  | 0,395 | 0,87  | 0,255 |       |       |       | 0,93  |
|          | T2 |       | 0,635 |       | 0,755 |       |       | 0,425 |       |
|          | T3 | 0,605 |       | 0,455 |       | 0,49  |       |       |       |
|          | T4 |       |       |       |       | 0,435 | 0,655 | 0,815 | 0,275 |

Figure 1: Approximation des temps de traitement produits / machines par l'espérance des lois uniformes

On calcule pour chaque machine  $M_j$  son débit asymptotique  $\rho_j$  défini par la somme des produits  $\lambda_i \mathbb{E}[S_{i,j}]$  charge induite par le produit  $i$  sur la machine  $j$ , comptée comme 0 si le produit  $i$  ne passe pas par la machine  $j$ .

- $\rho_1 = 0.29 * 0.68 + 0.47 * 0.605 = 0.48$
- $\rho_2 = 0.29 * 0.395 + 0.32 * 0.635 = 0.32$
- $\rho_3 = 0.29 * 0.87 + 0.47 * 0.455 = 0.47$
- $\rho_4 = 0.29 * 0.255 + 0.32 * 0.755 = 0.32$
- $\rho_5 = 0.47 * 0.49 + 0.38 * 0.435 = 0.39$
- $\rho_6 = 0.38 * 0.655 = 0.25$
- $\rho_7 = 0.32 * 0.425 + 0.38 * 0.815 = 0.446$
- $\rho_8 = 0.29 * 0.93 + 0.38 * 0.275 = 0.37$

La charge totale moyenne sur l'ensemble des machines est donc  $\rho_1 + \dots + \rho_8 = 3.05$

Une condition heuristique à l'existence d'un régime permanent est donc que **chaque employé soit capable d'absorber la charge moyenne des machines sur lesquelles il est responsable**. Pour l'instance I1, les ensembles machines/employés sont disjoints, de sorte qu'il suffit de sommer les charges des machines (1,2), (3,4), (5,6), (7,8) et de vérifier qu'elles sont bien  $< 1$ . Pour les instances I2, I3 et I4, certains employés ont une charge  $\rho > 1$  en faisant la simple somme, mais la mutualisation des tâches permet de répartir la charge entre les employés et d'assurer la convergence vers un régime permanent.

## Question 2

L'indicateur (i) est le plus pertinent ici. Il définit le temps de séjour moyen d'un produit dans le système. Il est donc sensible à la congestion et à l'attente entre deux traitements sur des machines. L'objectif est d'éviter que les produits restent bloqués longtemps dans le système, cet indicateur permet justement d'optimiser cet aspect.

L'indicateur (ii) est peu pertinent car en régime permanent stable, le débit de sortie d'un produit  $i$  est égal à son débit d'entrée  $\lambda_i$ . Il ne permet donc pas de mesurer la performance d'ALGO.

Le nombre moyen de produits dans l'atelier à tout instant (indicateur iii) est pertinent mais redondant avec l'indicateur (i) car d'après la loi de Little, la donnée  $s$  (temps dans le système) permet de connaître la donnée  $q$  (nombre d'éléments dans le système) car ils sont liés par le paramètre flux d'entrée  $\lambda$ .

## Question 3

Quelque soit ALGO, le produit doit subir tous ses temps de traitement. La borne est heuristique, on réutilise les moyennes de temps de traitement calculées en figure 1. Pour chaque type de produit, son temps de séjour est au moins aussi long de la somme des temps de traitement des machines par lesquelles ils transite.

$$\tau_1 \geq 0.68 + 0.395 + 0.87 + 0.255 + 0.93 = 3.13$$

$$\tau_2 \geq 0.635 + 0.755 + 0.425 = 1.815$$

$$\tau_3 \geq 0.605 + 0.455 + 0.49 = 1.55$$

$$\tau_4 \geq 0.435 + 0.655 + 0.815 + 0.275 = 2.18$$

La moyenne de ces temps, pondérée par les intensités d'arrivée donne une minoration heuristique du temps de séjour pour tout type de produit et n'importe quel algorithme:

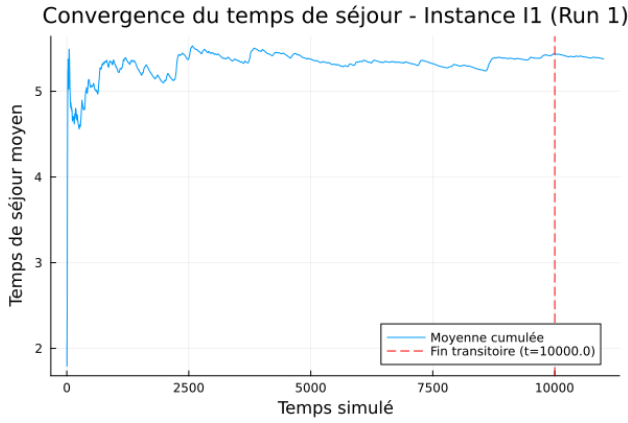
$$\tau \geq \frac{\lambda_1 \tau_1 + \lambda_2 \tau_2 + \lambda_3 \tau_3 + \lambda_4 \tau_4}{\lambda_1 + \lambda_2 + \lambda_3 + \lambda_4} = \mathbf{2.09}$$

## Question 4

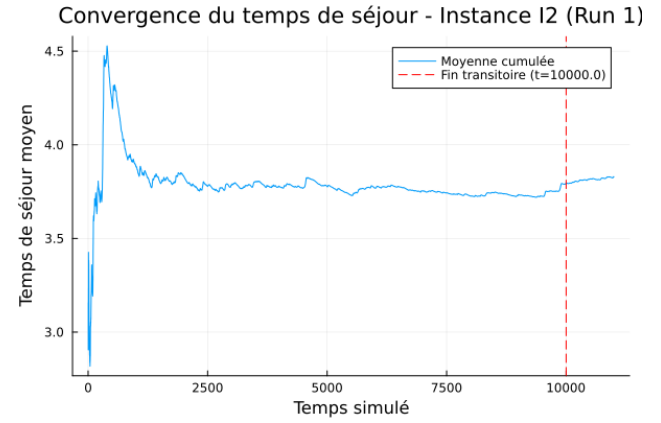
Implémentation du simulateur en Julia: Performances de l'algorithme FIFO sur les différentes instances

| Instance | Temps de séjour moyen | Occupation employés                      | Écart charge max/min |
|----------|-----------------------|------------------------------------------|----------------------|
| I1       | $5.59 \pm 0.24$       | 79.8%, 78.1%, 64.5%, 82.5%               | 18.0%                |
| I2       | $3.89 \pm 0.10$       | 73.9%, 75.7%, 74.4%, 80.6%               | 6.7%                 |
| I3       | $3.97 \pm 0.12$       | 61.2%, 29.1%, 40.9%, 76.2%, 32.5%, 62.8% | 47.1%                |
| I4       | $2.76 \pm 0.03$       | 49.2%, 49.4%, 54.5%, 54.1%, 44.6%, 51.1% | 9.9%                 |

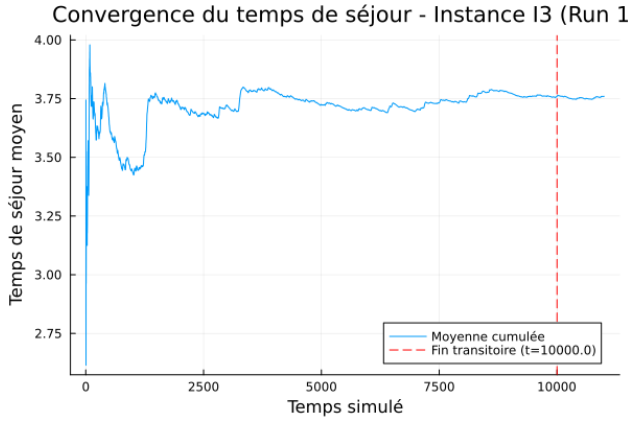
Table 1: Performances de l'algorithme FIFO sur les différentes instances



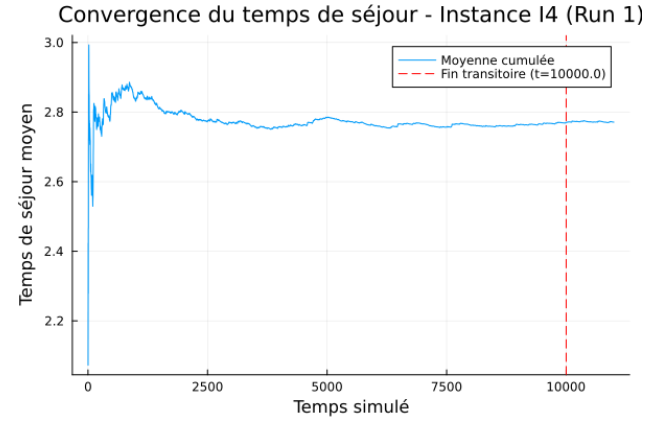
(a) Instance I1



(b) Instance I2



(c) Instance I3



(d) Instance I4

Figure 2: Convergence du temps de séjour moyen pour les différentes instances

## Question 5

Pour minimiser le temps de séjour moyen dans le système, on implémente l'algo **SPT (Shortest Processing Time)**. L'employé disponible regarde toutes les machines sur lesquelles il est qualifié et choisit celle avec un produit en attente ayant le plus petit temps de traitement. Le critère FIFO d'ancienneté est remplacé par un critère de durée de service.

SPT est le plus performant pour réduire le temps d'attente moyen car les tâches longues attendent un peu plus longtemps mais beaucoup de petites tâches sont traitées efficacement: la moyenne diminue. On peut prouver ce résultat:

On considère deux produits A et B de temps de traitement respectifs  $p_A$  et  $p_B$  avec  $p_A \leq p_B$

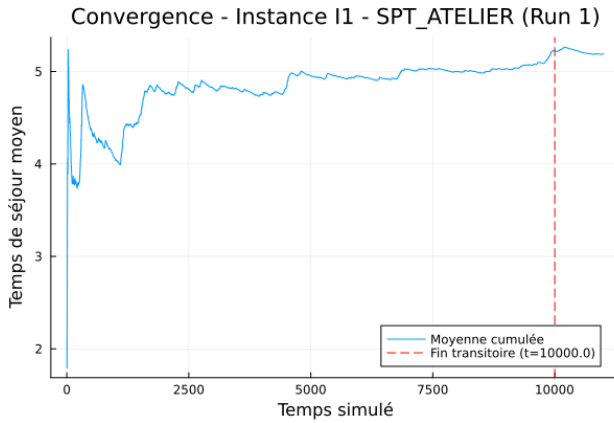
- Traiter A puis B (stratégie SPT): le temps de séjour total de A est  $S_A = p_A$  et celui de B est  $S_B = p_A + p_B$ .  
Moyenne des temps de séjour:  $s = \frac{2p_A + p_B}{2}$
- Traiter B puis A: le temps de séjour total de A est  $S_A = p_B + p_A$  et celui de B est  $S_B = p_B$ . Moyenne des temps de séjour:  $s = \frac{p_A + 2p_B}{2}$

Dans le deuxième cas, la moyenne des temps de séjour est plus longue. Le résultat se généralise pour n produits.

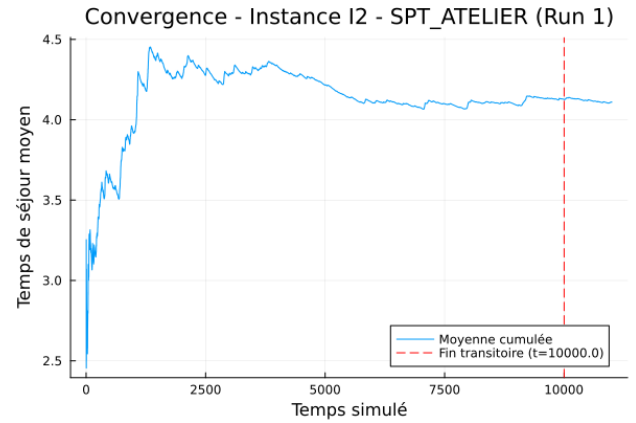
## Question 6: Implémentation Julia de l'ALGO SPT.

| Instance | Temps de séjour moyen | Occupation employés                      |
|----------|-----------------------|------------------------------------------|
| I1       | $4.86 \pm 0.14$       | 78.9%, 77.3%, 64.5%, 81.6%               |
| I2       | $4.02 \pm 0.12$       | 71.2%, 77.4%, 74.1%, 79.3%               |
| I3       | $3.68 \pm 0.09$       | 58.4%, 29.7%, 40.7%, 76.5%, 32.7%, 66.3% |
| I4       | $2.77 \pm 0.02$       | 49.4%, 49.1%, 55.9%, 54.7%, 44.2%, 52.2% |

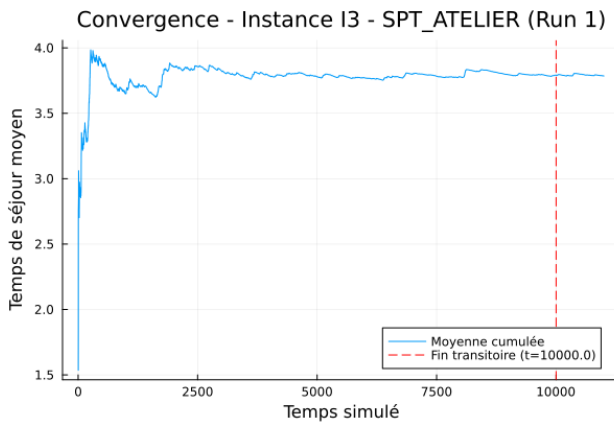
Table 2: Performances de l'algorithme SPT sur les différentes instances



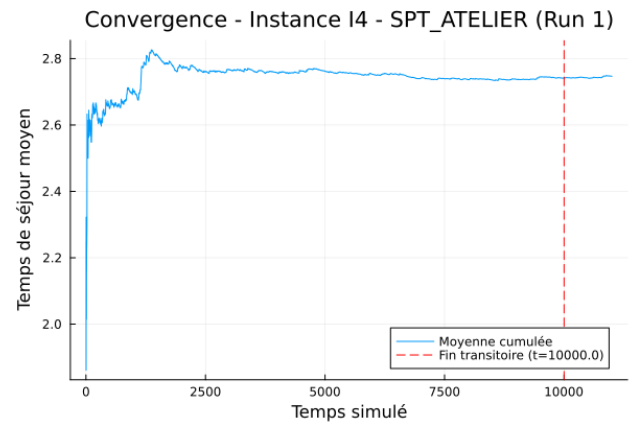
(a) Instance I1



(b) Instance I2



(c) Instance I3



(d) Instance I4

Figure 3: Convergence du temps de séjour moyen pour les différentes instances

L'algorithme SPT permet effectivement de globalement réduire le temps de séjour moyen avec une baisse d'environ 0.15 sur les instances I1, I3 et I4. SPT performe de manière équivalente à FIFO sur l'instance I2.

Une justification intuitive du manque d'amélioration sur I2: Dans l'instance I2 plusieurs produits sont traités par les mêmes machines, il y'a chevauchement des ressources; si ces machines ont de plus des temps de traitement courts alors il y'a un risque de trop les solliciter et de créer de la congestion dans le système. De plus, le système I2 en FIFO est déjà très chargé avec des taux d'occupation entre 73 et 80% et ce pour tous les ouvriers, ainsi la marge de progression est assez faible car les ouvrier travaillent déjà à quasi-plein temps.

## Question 7: Équilibrage de charge

On ne s'attend pas à une répartition équitable du travail car le système est structurellement déséquilibré (qualifications inégales, temps de traitement différents, taux d'arrivée différents etc.)

## Questions 8-9

### Nouvelle version d'ALGO pour équilibrage de la charge

Intuition: Lorsqu'un tâche sur une machine arrive: si l'employé est le seul à pouvoir la réaliser alors il la prend. Sinon, la tâche est laissée à l'employé le moins chargé capable de la réaliser. Ainsi, chaque employé agit sur les machines ou il est le plus indispensable et ça libère les autres machines pour des autres workers moins chargés.

---

#### Algorithm 1 Affectation de l'employé $E$ à une tâche

---

```

1:  $taches\_candidates \leftarrow []$ 
2: for all machine  $M$  telle que  $E$  est qualifié pour  $M$  do
3:   if  $M$  possède des produits en attente et  $M$  n'est pas occupée then
4:      $concurrents \leftarrow$  nombre d'employés dans la salle d'attente qualifiés pour  $M$ 
5:     ajouter  $(M, concurrents)$  à  $taches\_candidates$ 
6:   end if
7: end for
8:
9: if  $taches\_candidates$  est vide then
10:  envoyer  $E$  dans la salle d'attente
11: else
12:  trier  $taches\_candidates$  par  $concurrents$  croissant
13:  affecter  $E$  à la machine ayant le moins de concurrents
14: end if
```

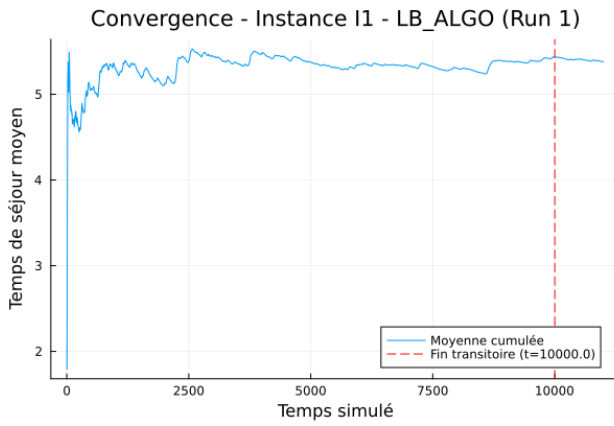
---

| Instance | Temps de séjour moyen | Occupation employés                      | Écart charge max/min |
|----------|-----------------------|------------------------------------------|----------------------|
| I1       | $5.59 \pm 0.24$       | 79.8%, 78.1%, 64.5%, 82.5%               | 18.0%                |
| I2       | $3.98 \pm 0.15$       | 73.8%, 76.8%, 74.6%, 80.7%               | 6.9%                 |
| I3       | $3.94 \pm 0.14$       | 60.7%, 29.6%, 40.6%, 77.2%, 33.0%, 64.1% | 47.6%                |
| I4       | $2.79 \pm 0.03$       | 49.0%, 49.6%, 55.0%, 54.5%, 44.7%, 51.6% | 10.3%                |

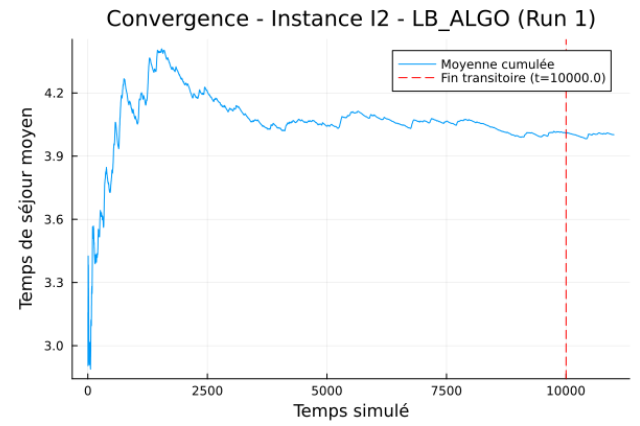
Table 3: Performances de l'algorithme ALGO avec équité améliorée sur les différentes instances

Le nouvel algorithme supposé améliorer l'équité entre les employés ne réduit pas l'écart entre les charges max et min (en comparaison avec l'implémentation FIFO). Le problème repose dans la structure des instances. Pour I1, les qualifications sont disjointes donc le nombre de concurrents est toujours 0 ou 1. Aucune amélioration n'est possible, d'où la différence de 18% inchangée. Sur les instances I2/I3/I4 le chevauchement existe mais l'employé n'a souvent qu'une seule machine candidate avec des produits en attente il n'y a pas de vrai choix à faire.

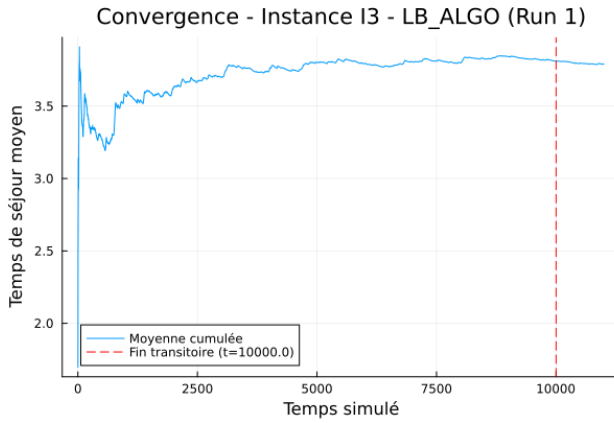
Conclusion: construire un ALGO agissant sur le choix des machines a peu de levier sur l'équité car les occasions de choix réel sont rares.



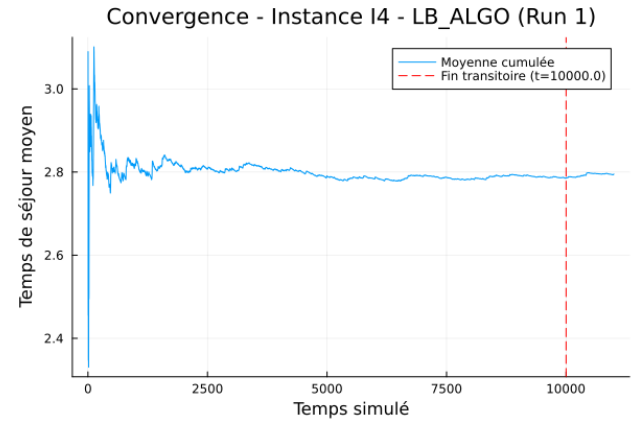
(a) Instance I1



(b) Instance I2



(c) Instance I3



(d) Instance I4

Figure 4: Convergence du temps de séjour moyen pour les différentes instances

## Nouvelle politique de choix d'employé dans la salle d'attente

Dans les trois implémentations précédentes (FIFO, SPT et ALGO équité+), le processus de sélection est le même et défini dans la fonction `choisir_employe_pour_machine`. On itère sur `file_inactifs` qui est triée par `temps_entree_salle`.

On change la stratégie de sélection depuis la salle d'attente vers une **Least Work First (LWF)**. On enregistre le `travail_cumule` à chaque step de la simulation et on choisit l'employé qualifié ayant le moins de travail cumulé. À chaque fois qu'une machine se libère et qu'il y a compétition entre plusieurs employés qualifiés, on choisit systématiquement le sous-chargé. Sur le long terme, les employés les plus chargés passent plus de temps en salle d'attente pendant que leurs collègues rattrapent leur retard et la charge doit en théorie se répartir. Pour que cette stratégie fonctionne, il faut qu'il y ait régulièrement plusieurs employés qualifiés disponibles en salle d'attente pour la même machine; donc il faut un chevauchement de qualifications. Pour I1 où les qualifications sont disjointes, on anticipe que LWF ne changera rien.

On implémente cette nouvelle stratégie de salle d'attente par dessus l'implémentation FIFO du simulateur. Le changement est dans la fonction `choisir_employe_pour_machine`:

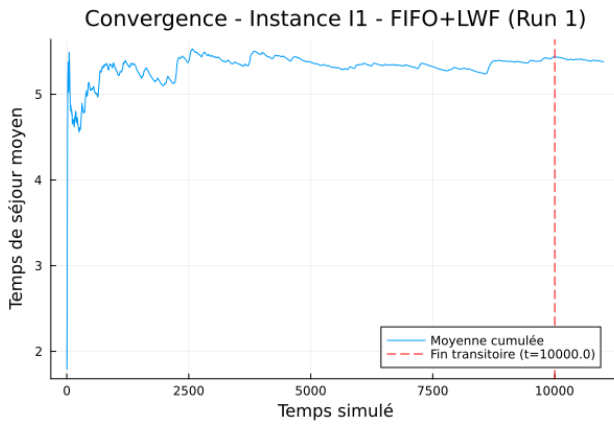
Implémentation FIFO:

```
for emp_id in atelier.file_inactifs # trié par temps_entree_salle
  if machine_id in emp.qualifications ...
```

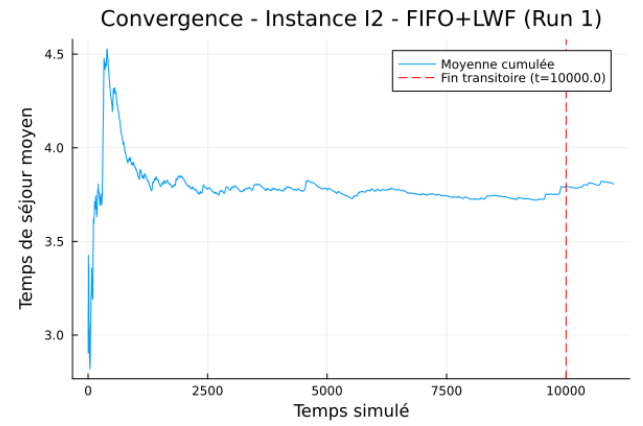
Implémentation LWF:

```
candidats = [id for id in atelier.file_inactifs
              if machine_id in atelier.employees[id].qualifications]

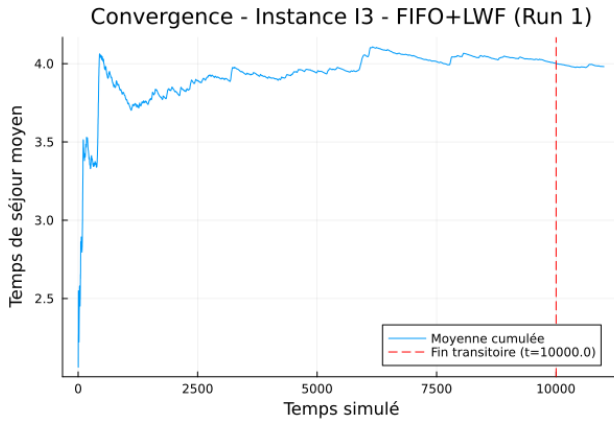
emp_id = argmin(id -> atelier.employees[id].travail_cumule, candidats)
```



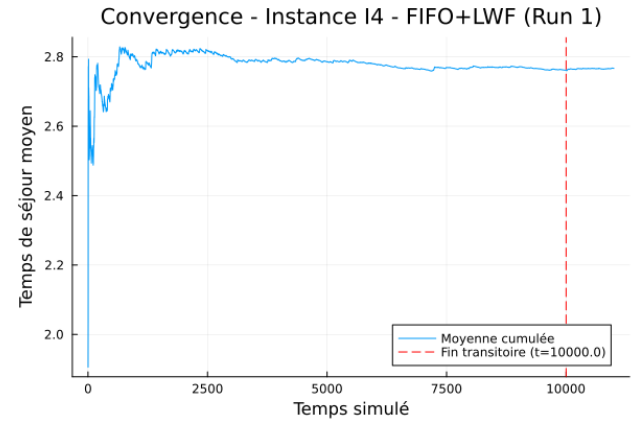
(a) Instance I1



(b) Instance I2



(c) Instance I3



(d) Instance I4

Figure 5: Convergence du temps de séjour moyen pour les différentes instances

| Instance | Temps de séjour moyen | Occupation employés                      | Écart charge max/min |
|----------|-----------------------|------------------------------------------|----------------------|
| I1       | $5.59 \pm 0.24$       | 79.8%, 78.1%, 64.5%, 82.5%               | 18.0%                |
| I2       | $3.97 \pm 0.13$       | 76.0%, 75.7%, 75.4%, 77.1%               | 1.6%                 |
| I3       | $3.96 \pm 0.10$       | 62.7%, 34.1%, 35.5%, 77.2%, 34.2%, 62.9% | 43.1%                |
| I4       | $2.79 \pm 0.02$       | 50.1%, 51.1%, 51.1%, 51.1%, 50.9%, 51.1% | 0.2%                 |

Table 4: Performances de l'algorithme FIFO avec sélection LWF des employés dans la salle d'attente sur les différentes instances

Comme anticipé, il n'y a aucun effet sur l'instance I1 ou aucune levier n'est actionnable car les employés ont des compétences disjointes. Les instances I2 et I4, qui ont le plus de chevauchements de compétences ont un écart max/min bien meilleur. On passe de 6.7% à 1.6% pour I2 et 9.9% à 0.2% pour l'instance I4. Sur cette dernière instance les charges sont presque parfaitement équilibrées. L'instance I3 elle possède moins de compétences communes entre les employés, elle bénéficie donc d'une baisse de 5% sur sa différence max/min.

On notera également que cette nouvelle stratégie de sélection de l'employé dans la salle d'attente ne change pas le temps de séjour moyen, sinon l'améliore légèrement.

En conclusion, agir sur le choix des employés plutôt que le choix des machines a posteriori est le meilleur levier pour mieux répartir la charge de travail entre eux. Le plus les ouvriers ont des compétences en commun, le meilleure l'amélioration sera.

## Question 10: Le critère max-min

Le critère proposé pour quantifier la distribution de charge est l'écart max-min, i.e. la différence entre le taux d'occupation de l'employé le plus chargé et celui du moins chargé. Ce critère permet de mesurer l'équité mais possède des limites:

- Il ignore la distribution intermédiaire. On pourrait avoir une distribution égale sauf pour un seul employé (ex: 90, 90, 90, 50) ou bien une distribution répartie plus linéairement (90, 80, 70, 50). Chaque situation donne un même écart de 30 mais les distributions sont inégales. Pour y remédier on pourrait mesurer La variance ou l'écart-type des occupations.
- Le critère est sensible aux cas extrêmes. Un seul employé très sous chargé (par exemple si ses qualifications couvrent des machines peu sollicitées) peut faire exploser le critère, même si tous les autres sont parfaitement équilibrés. L'écart est alors très dépendant de la structure des instances plus que de la qualité de l'algorithme.
- Il n'est pas normalisé et donc un écart de 2% sur des gros pourcentages d'occupation (90,80 etc) n'a pas la même signification qu'un écart sur des petites valeurs (10,20 etc).

On pourrait considérer d'autres métriques comme **l'écart-type des occupations** qui prend en compte la structure interne de système et non seulement les deux extrêmes.